



Universidad
Nacional
de Loja

UNIVERSIDAD NACIONAL DE LOJA

FACULTAD DE ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES

NO RENOVABLES

CARRERA DE COMPUTACIÓN

MATEMATICAS DISCRETAS

TITULO DE LA PRÁCTICA:

**“PROPOSICIONES, CONECTORES LÓGICOS, INFERENCIA, DEDUCCIÓN Y TABLAS
DE VERDAD”**

ESTUDIANTES:

David Luna

Andy Ordoñez

Miguel Rojas

Karen Lalangui

Docente: Ing. Mario Cueva

Fecha: 28/04/2026

REPORTE TÉCNICO DE ACTIVIDADES

PRACTICO-EXPERIMENTALES Nro. 1

1. DATOS GENERALES

Nombre del estudiante(s)	Miguel Rojas, David Luna, Andy Ordoñez y Karen Lalangui.
Asignatura	Matemáticas discretas
Ciclo	1er ciclo
Unidad	Unidad 1: Lógica Matemática
Resultado de aprendizaje de la unidad	Desarrolla habilidades de razonamiento lógico para solucionar problemas ligados a la ingeniería, bajo los principios de, solidaridad, transparencia, responsabilidad y honestidad
Práctica Nro.	001 FASE 2
Título de la Práctica	Proposiciones, Conectores Lógicos, inferencia, deducción y tablas de verdad.
Nombre del Docente	Ing. Mario Enrique Cueva Hurtado
Fecha	06 de abril al 15 de mayo de 2026
Horario	Lunes, martes, jueves 07:30 – 09:30
Lugar	Aula de la carrera de Computación
Tiempo planificado en el Sílabo	18 oras (3 horas por sección APE)

2. OBJETIVO(S) DE LA PRÁCTICA

- ✓ Identificar y clasificar proposiciones lógicas simples y compuestas, distinguiendo entre proposiciones y enunciados no proposicionales.
- ✓ Comprender y aplicar los conectores lógicos fundamentales (negación, conjunción, disyunción, condicional, bicondicional,) en la formulación de expresiones lógicas.
- ✓ Construir tablas de verdad para proposiciones compuestas, determinando su valor de verdad en todos los casos posibles.

- ✓ Aplicar las leyes de inferencia y deducción lógica para resolver problemas de razonamiento lógico en contextos computacionales y de ingeniería.
- ✓ Desarrollar habilidades de trabajo colaborativo, pensamiento crítico y resolución de problemas, integrando principios de equidad, inclusión y responsabilidad ética en el aprendizaje.

3. MATERIALES, EQUIPOS Y HERRAMIENTAS

- ✓ Hojas de trabajo impresas con ejercicios de lógica proposicional y tablas de verdad.
- ✓ Guía de ejercicios de inferencia y deducción lógica

4. PROCEDIMIENTO

Fase 2: Construcción de las tablas de verdad y operaciones combinadas

- ✓ Práctica grupal: cada equipo resuelve ejercicios de simplificación de expresiones lógicas aplicando las leyes fundamentales (ley de identidad, complemento, idempotencia, conmutativa, asociativa, distributiva, de Morgan, absorción).
- ✓ Los equipos analizan problemas de lógica computacional: cómo las expresiones booleanas se aplican en la evaluación de condiciones en programación (if-else, while, operadores lógicos C).

5. RESULTADOS

Ejercicios resueltos

$$1. [(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$$

$$[(p \vee q \vee r) \vee \neg(p \vee q \vee r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow \text{Ley de Morgan.}$$

$$\vee \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow \text{ley complemento.}$$

$$[q \vee (r \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r) \rightarrow \text{Ley distributiva}$$

$$[q \vee (\vee) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r) \rightarrow \text{ley complemento}$$

$$[q \vee (\neg q \wedge r)] \leftrightarrow (q \vee r) \rightarrow \text{distributiva}$$

$$[(q \vee \neg q) \wedge (q \vee r)] \leftrightarrow (q \vee r) \rightarrow \text{distributiva}$$

$V \wedge (q \vee r) \leftrightarrow (q \vee r) \rightarrow$ ley complemento.

$(q \vee r) \leftrightarrow (q \vee r) \rightarrow$ Ley de idempotencia.

$(q \vee r)$ Rta.

2. $\{\sim [\sim (\sim p \rightarrow \sim q) \wedge \sim (p \wedge \sim r)] \vee \sim (p \rightarrow r)\}$

$\sim [\sim (p \vee \sim q) \wedge \sim (p \wedge \sim r)] \vee \sim (\sim p \vee r) \rightarrow$ ley del condicional (2 veces)

$(p \vee \sim q) \vee (p \wedge \sim r) \vee (p \wedge \sim r) \rightarrow$ ley de Morgan (2 veces)

$(p \vee \sim q) \vee (p \wedge \sim r) \rightarrow$ ley de idempotencia

$\sim q \vee [p \vee (p \wedge \sim r)] \rightarrow$ ley asociativa

$(\sim q \vee p) \rightarrow$ ley de absorción

$(p \vee \sim q) \rightarrow$ Ley conmutativa

3. $r \wedge (\sim t \wedge r)$

$r \wedge (r \wedge \sim t) \rightarrow$ Propiedad Conmutativa

$(r \wedge r) \wedge \sim t \rightarrow$ Propiedad Asociativa

$r \wedge \sim t \rightarrow$ Idempotencia

4. $\sim (p \rightarrow t) \vee [(r \wedge \sim q) \vee (p \rightarrow t)]$

$\sim (p \rightarrow t) \vee [(p \rightarrow t) \vee (r \wedge \sim q)] \rightarrow$ Conmutativa

$[\sim (p \rightarrow t) \vee (p \rightarrow t)] \vee (r \wedge \sim q) \rightarrow$ Asociativa

$V \vee (r \wedge \sim q) \rightarrow$ Idempotencia

$V \rightarrow$ Rta

5. $[p \wedge \sim (\sim q)] \wedge \sim q$

$[p \wedge q] \wedge \sim q \rightarrow$ Doble Negación

$p \wedge [q \wedge \sim q] \rightarrow$ Propiedad Asociativa

$p \wedge F \rightarrow$ Leyes de Negación

$F \rightarrow$ Leyes de Identidad

¿Cómo las expresiones booleanas se aplican en la evaluación de condiciones en programación (if-else, while, operadores lógicos en C)?

En C, históricamente, no existía el tipo de dato booleano (True / False). C resuelve la lógica usando matemáticas simples con números enteros:

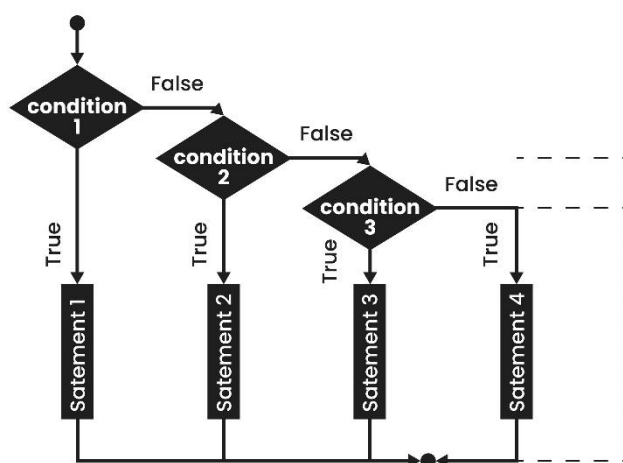
- ✓ **Falso:** Se evalúa estrictamente como el número 0.
- ✓ **Verdadero:** Se evalúa como cualquier valor distinto de cero (generalmente 1, pero valores como -5 o 100 también son lógicamente verdaderos).

Nota: En estándares modernos (C99 en adelante), puedes importar la librería `<stdbool.h>` para usar `bool`, `true` y `false`, lo cual hace el código más legible, pero por debajo siguen siendo 1 y 0.

Evaluando condiciones con if-else (Bifurcación)

Una expresión booleana en un bloque if actúa como un interruptor en las vías de un tren: decide qué camino tomará el programa. La CPU evalúa la expresión; si da un resultado distinto de 0, entra al bloque if. Si da 0, se desvía al bloque else.

IF-ELSE-IF STATEMENT



Ejemplo

```
C If-else.c > main()
1  #include <stdio.h>
2  int main(){
3
4      int temperatura = 25;
5      int lluvia = 1;
6      //el 1 representa verdadero(esta lloviendo)
7
8      if (temperatura > 20 && lluvia == 0) {
9          printf("Hace buen clima para ir al parque.\n");
10     } else {
11         printf("Major nos quedamos en casa.\n");
12     }
13
14     return 0;
15 }
16
17
18
19
20
21
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\valen\Desktop\PRIMER CICLO\Programaciòn\Programa C> gcc .\if-else.c -o if-else
PS C:\Users\valen\Desktop\PRIMER CICLO\Programaciòn\Programa C> .\if-else.exe
Major nos quedamos en casa.
PS C:\Users\valen\Desktop\PRIMER CICLO\Programaciòn\Programa C> 
```

Lógica computacional aplicada: El programa evalúa $(25 > 20) \rightarrow$ Verdadero (1). Luego evalúa $(1 == 0) \rightarrow$ Falso (0). Finalmente, calcula $1 \text{ AND } 0$, lo cual resulta en 0 (Falso). El flujo salta al else.

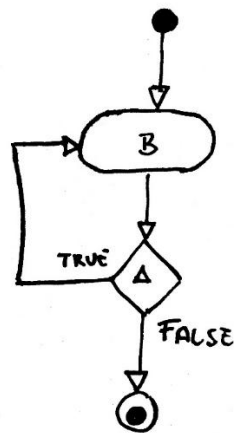
Evaluando condiciones con while (Iteración)

El bucle while utiliza la lógica booleana para decidir si debe repetir un bloque de código o detenerse. La condición se evalúa antes de cada ciclo. Mientras la expresión siga siendo Verdadera (no cero), el bucle continúa.

```

DO B
WHILE (A)
END WHILE

```



Ejemplo

```

19  #include <stdio.h>
20
21  int main() {
22      int energia = 3;
23
24      // Mientras la energía sea "verdadera" (mayor que 0)
25      while (energia > 0) {
26          printf("Personaje corriendo. Energia restante: %d\n", energia);
27          energia--; // Reducimos la energía en 1
28      }
29      printf("El personaje se ha detenido por fatiga.\n");
30      return 0;
31  }
32
33
34

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\valen\Desktop\PRIMER CICLO\Programaciòn\Programa C> gcc .\if-else.c -o if-else
PS C:\Users\valen\Desktop\PRIMER CICLO\Programaciòn\Programa C> .\if-else.exe
Major nos quedamos en casa.
PS C:\Users\valen\Desktop\PRIMER CICLO\Programaciòn\Programa C> gcc .\if-else.c -o if-else
PS C:\Users\valen\Desktop\PRIMER CICLO\Programaciòn\Programa C> .\if-else.exe
Personaje corriendo. Energia restante: 3
Personaje corriendo. Energia restante: 2
Personaje corriendo. Energia restante: 1
El personaje se ha detenido por fatiga.
PS C:\Users\valen\Desktop\PRIMER CICLO\Programaciòn\Programa C> 

```

Lógica computacional aplicada: El bucle se repetirá cuando energía sea 3, 2 y 1. Cuando energía llega a 0, la expresión $0 > 0$ se evalúa como 0 (Falso). El bucle se rompe y el programa continúa.

6. PREGUNTAS DE CONTROL

Explique la diferencia entre una proposición simple y una proposición compuesta. ¿Por qué es importante distinguir entre ambas en el contexto de la programación y diseño de algoritmos? Argumente con ejemplos concretos en lenguajes de programación.

Las proposiciones simples son aquellas formadas por una sola afirmación (V, F). Mientras que, las proposiciones compuestas son aquellas formadas por proposiciones simples, y están conectadas a través de conectores lógicos ($\wedge, \vee, \neg, \rightarrow, \leftrightarrow$).

Un programa entiende condiciones no ideas, es importante distinguir entre las dos proposiciones para identificar correctamente las condiciones y evitar errores lógicos dentro del algoritmo.

Ejemplo:

```
C ejemplo.c > main()
1  #include <stdio.h>
2
3  int main(){
4
5      int edad = 20;
6      int tieneID = 1;
7
8      if (edad >= 18 && tieneID == 1) {
9          printf("Acceso permitido\n");
10     } else {
11         printf ("Acceso denegado\n");
12     }
13
14     return 0;
15
16 }
17
18
19
20
21
22
23
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\valen\Desktop\PRIMER CICLO\Programaciòn\Programa C> gcc .\ejemplo.c -o ejemplo
PS C:\Users\valen\Desktop\PRIMER CICLO\Programaciòn\Programa C> .\ejemplo.exe
Acceso permitido
PS C:\Users\valen\Desktop\PRIMER CICLO\Programaciòn\Programa C>
```


7. BIBLIOGRAFÍAS

Microsoft Build 2026. (3 de junio de 2026). Microsoft. Obtenido de <https://learn.microsoft.com/es-es/cpp/c-language/c-logical-operators?view=msvc-170>

8. ANEXOS

✓ Karen Lalangui

REPORTE TÉCNICO DE ACTIVIDADES
PRÁCTICO EXPERIMENTALES NEOL

RESULTADOS

1. $[(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$

$[(p \vee q \vee r) \vee (\neg p \vee q \vee r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow$ ley de Morgan

$\vee \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow$ ley de negación

$[q \vee (r \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow$ ley distributiva

$[q \vee \vee (\neg q \wedge r)] \rightarrow$ ley de negación

$[q \vee (\neg q \wedge r)] \rightarrow$ identidad

$(q \vee \neg q) \wedge (q \vee r) \rightarrow$ distributiva

$\vee \wedge (q \vee r) \rightarrow$ ley de negación

$(q \vee r) \leftrightarrow (q \vee r)$

2. $[\neg [\neg (\neg p \rightarrow \neg q) \wedge \sim (p \wedge \neg r)] \vee \sim (p \rightarrow r)]$

$\neg [\neg (p \vee \neg q) \wedge \neg (p \wedge \neg r)] \vee \neg (\neg p \vee r) \rightarrow$ condicional 2 veces.

$(p \vee \neg q) \vee (p \wedge \neg r) \vee (p \wedge \neg r) \rightarrow$ ley Morgan 2x

$(p \vee \neg q) \vee (p \wedge \neg r) \rightarrow$ ley idempotencia

$\neg \neg q \vee [p \vee (p \wedge \neg r)] \rightarrow$ ley asociativa m. l. y de

$(\neg q \vee p) \rightarrow$ ley absorción

$(p \vee \neg q) \rightarrow$ conmutativa.

$$3. r \wedge (\sim t \wedge r)$$

$$r \wedge (r \wedge \sim t) \rightarrow \text{Conmutativa}$$

$$(r \wedge r) \wedge \sim t \rightarrow \text{asociativa}$$

$$r \wedge \sim t \rightarrow \text{Idempotencia}$$

$$4. \sim(p \rightarrow t) \vee [(r \wedge \sim q) \vee (p \rightarrow t)]$$

$$\sim(p \rightarrow t) \vee [(p \rightarrow t) \vee (r \wedge \sim q)] \rightarrow \text{Conmutativa}$$

$$[\sim(p \rightarrow t) \vee (p \rightarrow t)] \vee (r \wedge \sim q) \rightarrow \text{Asociativa}$$

$$\vee \vee (r \wedge \sim q) \rightarrow \text{Idempotencia}$$

$$\boxed{\vee} \rightarrow$$

$$5. [p \wedge \neg(\neg q)] \wedge \neg q$$

$$[p \wedge q] \wedge \neg q \rightarrow \text{ley de negación doble.}$$

$$p \wedge (q \wedge \neg q) \rightarrow \text{Asociativa}$$

$$p \wedge F \rightarrow \text{ley de negación}$$

$$F \rightarrow \text{leyes de identidad}$$

Investigación

1. ¿Como las expresiones booleanas se aplican en la evaluación de condiciones en programación (if-else, while, operadores lógicos en C)?

En C, históricamente, no existía el tipo de dato booleano (True / false). "C" resuelve la lógica usando matemáticas simples como números enteros

✓ falso: Se evalúa estrictamente con el valor 0

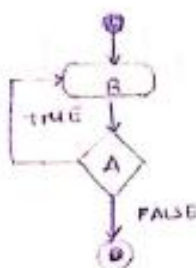
✓ Verdadero: Se evalúa con cualquier valor distinto de cero (generalmente 1)

nota:

Una expresión booleana en un bloque if actúa como un interruptor en las vías del tren, decide el camino que tomará el programa. La CPU evalúa la expresión; si da un resultado distinto de 0, entra al bloque if. Si da 0 se deriva al bloque else.

EVALUANDO CONDICIONES CON WHILE (ITERACIÓN).

El bucle while utiliza la lógica booleana para decidir si debe repetir un bloque de código o detenerse. La condición se evalúa antes de cada ciclo. Mientras la expresión siga siendo verdadera (no 0), el bucle continúa.



PREGUNTAS DE CONTROL.

Explique la diferencia entre una proposición simple y una proposición compuesta.
¿Por qué es importante distinguir entre ambas en un contexto de la programación y diseño de algoritmo?

Las proposiciones simples son aquellas formadas por una sola afirmación (V, F) mientras que las proposiciones compuestas son aquellas formadas por proposiciones simples y están conectadas a través de conectivos lógicos ($\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$, $\neg$).

Un programa entiende condiciones, no ideas, es importante distinguir entre ellas para identificar correctamente las condiciones y evitar errores lógicos dentro del algoritmo.

✓ Andy Ordoñez

Ejercicios resueltos

$$1. [(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \\ \leftrightarrow (q \vee r)$$

$$[(p \vee q \vee r) \vee \neg(p \vee q \vee r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \\ \rightarrow \text{Ley de Morgan}$$

$$\vee \wedge [(q \wedge r) \vee (p \vee \neg r) \vee (\neg q \wedge r)] \rightarrow \text{Ley complemento}$$

$$[q \vee (r \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r) \rightarrow \text{Ley distributiva}$$

$$[q \vee (\vee) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r) \rightarrow \text{Ley complemento}$$

$$[q \vee (\neg q \wedge r)] \leftrightarrow (q \vee r) \rightarrow \text{Ley distributiva}$$

$$[(q \vee \neg q) \wedge (q \vee r)] \leftrightarrow (q \vee r) \rightarrow \text{Ley distributiva}$$

$$\vee \wedge (q \vee r) \leftrightarrow (q \vee r) \rightarrow \text{Ley complemento}$$

$$(q \vee r) \leftrightarrow (q \vee r) \rightarrow \text{Ley de idempotencia}$$

$$(q \vee r) \text{ Rta.}$$

$$2. \{ \sim [\sim (\sim p \rightarrow \sim q) \wedge \sim (p \wedge \sim r)] \vee \sim (p \rightarrow r) \}$$

$$\sim [\sim (p \vee \sim q) \wedge \sim (p \wedge \sim r)] \vee \sim (p \vee r) \rightarrow \text{Ley del condicional (x2)}$$

$$(p \vee \sim q) \vee (p \wedge \sim r) \vee (p \wedge \sim r) \rightarrow \text{Ley de Morgan (x2)}$$

$$(p \vee \sim q) \vee (p \wedge \sim r) \rightarrow \text{Ley de idempotencia}$$

$$\sim q \vee [p \vee (p \wedge \sim r)] \rightarrow \text{Ley asociativa}$$

$$(\sim q \vee p) \rightarrow \text{Ley de absorción}$$

$$(p \vee \sim q) \rightarrow \text{Ley conmutativa}$$

$$3. r \wedge (\sim t \wedge r)$$

$$r \wedge (r \wedge \sim t) \rightarrow \text{Propiedad conmutativa}$$

$$(r \wedge r) \wedge \sim t \rightarrow \text{Propiedad Asociativa}$$

$$4. \sim(p \rightarrow t) \vee [(r \wedge \sim q) \vee (p \rightarrow t)]$$

$$\sim(p \rightarrow t) \vee [(p \rightarrow t) \vee (r \wedge \sim q)] \rightarrow \text{conmutativa}$$

$$[\sim(p \rightarrow t) \vee (p \rightarrow t)] \vee (r \wedge \sim q) \rightarrow \text{Asociativa}$$

$$\vee \vee (r \wedge \sim q) \rightarrow \text{Idempotencia}$$

$$\vee \rightarrow \text{Idempotencia}$$

$$5. [p \wedge \sim(\sim q)] \wedge \sim q$$

$$[p \wedge q] \wedge \sim q \rightarrow \text{Doble Negación}$$

$$p \wedge [q \wedge \sim q] \rightarrow \text{Propiedad Asociativa}$$

$$p \wedge F \rightarrow \text{Leyes de negación}$$

$$F \rightarrow \text{Leyes de Idempotencia}$$

¿Cómo las expresiones booleanas se aplican en la evaluación de condiciones en programación (if-else, while, operadores lógicos en C)?
En C, históricamente, no existía el tipo de dato booleano (True/False). C resuelve la lógica usando matemáticas simples números enteros:

✓ Falso. Se evalúa estrictamente como el número 0.

✓ Verdadero. Se evalúa como cualquier valor distinto de cero.

Evaluando condiciones con if-else (Bifurcación).

Una expresión booleana en un bloque if actúa como un interruptor en las vías de un tren: decide qué camino tomará el programa. La CPU evalúa la expresión; si da un resultado distinto de 0, entra al bloque if; si da 0, se desvía al bloque else.

Lógica computacional aplicada: El programa evalúa $(25 > 20) \rightarrow \text{Verdadero (1)}$. Luego evalúa $(1 == 0) \rightarrow \text{Falso (0)}$. Finalmente, calcula $1 \text{ AND } 0$, lo cual resulta en 0 (Falso). El flujo salta al else.

Evaluando condiciones con while (Iteración).

El bucle while utiliza la lógica booleana para decidir si debe repetir un bloque de código o

detenerse. La condición se evalúa antes de cada ciclo. Mientras la expresión siga siendo verdadera (no cero), el bucle continúa.

Lógica computacional aplicada. El bucle se repetirá cuando energía sea 3, 2 y 1. Cuando energía llega a 0, la expresión $0 > 0$ se evalúa como 0 (falso). El bucle se rompe y el programa continúa.

Pregunta de control

Explique la diferencia entre una proposición simple y una proposición compuesta. ¿Por qué es importante distinguir entre ambas en el contexto de la programación y diseño de algoritmos? Argumente con ejemplos concretos en lenguajes de programación. Las proposiciones simples son aquellas formadas por una sola afirmación (V, F). Mientras que, las proposiciones compuestas son aquellas formadas por proposiciones simples, y están conectadas a través de conectores lógicos ($\neg, \vee, \wedge, \rightarrow, \leftrightarrow$).

Un programa entiende condiciones no ideas, es importante distinguir entre las dos proposiciones para identificar correctamente las condiciones y evitar errores lógicos dentro del algoritmo.

✓ Santiago Luna

Trabajo Grupal 2.

Nombre: David Luna Lara

Fecha: 17/04/2026

Ciclo: 1 "A"

Carera: Computación

Actividades

1. Ejercicios resueltos

$$\bullet [(p \vee q \vee r) \vee (\neg p \wedge q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow (q \vee r)$$

$$[(p \vee q \vee r) \vee (\neg p \vee q \vee r)] \wedge [(q \wedge r) \vee (q \wedge \neg r)] \rightarrow \text{Ley de Morgan.}$$

$$\vee \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow \text{ley complemento}$$

$$[q \vee (\neg \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow Cq \vee r \rightarrow \text{ley distributiva.}$$

$$[q \vee (\vee) \vee (\neg q \wedge r)] \rightarrow Cq \vee r \rightarrow \text{ley complemento.}$$

$$[q \vee (\neg q \wedge r)] \rightarrow Cq \vee r \rightarrow \text{distributiva.}$$

$$[Cq \vee \neg q] \wedge (q \vee r) \rightarrow Cq \vee r \rightarrow \text{distributiva.}$$

$$\vee \wedge Cq \vee r \rightarrow Cq \vee r \rightarrow \text{ley complemento.}$$

$$Cq \vee r \rightarrow Cq \vee r \rightarrow \text{ley de idempotencia.}$$

$$Cq \vee r) \text{ Rta.}$$

$$\bullet [-[-(-p \rightarrow \neg q) \wedge \neg(p \wedge \neg r)] \vee \neg p \rightarrow r]$$

$$\neg [\neg(p \vee \neg q) \wedge \neg(p \wedge \neg r)] \vee \neg(p \vee r) \rightarrow \text{ley del condicional (2 veces)}$$

$$(p \vee \neg q) \vee (p \wedge \neg r) \vee (p \wedge \neg r) \rightarrow \text{ley de Morgan (2 veces)}$$

$$Cp \vee \neg q) \vee (p \wedge \neg r) \rightarrow \text{ley de idempotencia.}$$

$$\neg q \vee [p \vee (p \wedge \neg r)] \rightarrow \text{ley asociativa}$$

$$(\neg q \vee p) \rightarrow \text{ley de absorción}$$

$$Cp \vee \neg q) \rightarrow \text{ley conmutativa.}$$

$$\bullet r \wedge (\neg \neg \wedge r).$$

$\forall x (x \wedge \neg x) \rightarrow$ Propiedad Conmutativa.

$(x \wedge y) \wedge z \rightarrow$ Propiedad Asociativa.

$x \wedge \neg x \rightarrow$ Identidad.

• $\neg(p \rightarrow \neg) \vee [(x \wedge \neg q) \vee (p \rightarrow \neg)]$

$\neg(p \rightarrow \neg) \vee [(p \rightarrow \neg) \vee (x \wedge \neg q)] \rightarrow$ Conmutativa.

$[\neg(p \rightarrow \neg) \vee (p \rightarrow \neg)] \vee (x \wedge \neg q) \rightarrow$ Asociativa.

$\forall x (x \wedge \neg q) \rightarrow$ Idempotencia

$\forall x R \neg q$

• $[p \wedge \neg(q)] \wedge \neg q$

$[p \wedge q] \wedge \neg q \rightarrow$ Doble Negación

$p \wedge [q \wedge \neg q] \rightarrow$ Propiedad Asociativa

$p \wedge F \rightarrow$ leyes de Negación

$F \rightarrow$ leyes de Idempotencia

1. La verdad y falsedad en C: Todo son números.

El primer concepto de lógica computacional que debería entender en C es que, históricamente, no existía el tipo de dato booleano (True/False). C resuelve lógicamente usando matemáticos simples con números enteros:

- Falso: Se evalúa estrictamente como el número 0.
- Verdadero: Se evalúa como cualquier valor distinto de cero.

2. Evaluando condiciones con if - else

Una expresión booleana en un bloque if actúa como un interruptor en las vías de un tren: decide qué camino tomará el programa. La CPU evalúa la expresión; si da un resultado distinto de 0, entra al bloque if. Si da 0, se desvía al bloque else.

3. Ejemplo de análisis lógico: C

```
#include <stdio.h>
```

```
int main() {
```

```
    int temperatura = 25;
```

```
    int lluvia = 1; // 1 representa verdadero
```

```
    if (temperatura > 20 && lluvia == 0) {
```

```
        printf("Hace buen clima para ir al parque.\n");
```

```
    } else {
```

```
        printf("Mejor nos quedamos en casa.\n");
```

```
    }
```

```
    return 0;
```

```
}
```

4. Evaluando condiciones con while

El bucle while utiliza la lógica booleana para decidir si debe repetir un bloque de código o detenerse. La condición se evalúa antes de cada ciclo. Mientras la expresión siga siendo Verdadera (no cero) el bucle continúa.

5. Ejemplo de análisis lógico C

```
* include <stdio.h>
```

```
int main() {
```

```
    int energia = 3;
```

```
    // mientras la energía sea "verdadera" (mayor que 0)
```

```
    while (energia > 0) {
```

```
        printf("Personaje corriendo. Energía restante: %d/n", energia);
```

```
        energia --; // reducimos la energía en 1
```

```
    }
```

```
    printf("El personaje se ha detenido por fatiga.\n");
```

```
    return 0;
```

```
}
```

6. Un problema clásico de lógica en C: El Cortocircuito

Una regla de optimización crítica en la lógica de C es la evaluación de cortocircuito. C evalúa las expresiones lógicas complejas de izquierda a derecha y se detiene en el instante en que ya sabe el resultado final.

- Con AND: si la primera condición es 0, C ignora y no ejecuta

- Con OR: si la primera condición es 1, C ignora y no ejecuta.

Regunta de Control.

- Explique la diferencia entre una proposición simple y una proposición compuesta.
 - ¿Por qué es importante distinguir entre ambos en el contexto de la programación y diseño de algoritmo? Argumente con ejemplos concretos en lenguajes de programación.
- Las proposiciones simples son aquellas formadas por una sola afirmación (V, F). Mientras que las proposiciones compuestas son aquellas formadas por proposiciones simples, y están conectadas a través de conectores lógicos ($\neg$, $\vee$, $\wedge$, $\rightarrow$, $\leftrightarrow$).

Un programa entiende condiciones no ideas, es importante distinguir entre las dos proposiciones para identificar correctamente las condiciones y evitar errores lógicos dentro del algoritmo.

Ejemplo.

```
# Include <stdio.h>
int main() {
    int
    int edad = 20;
    int tieneID = 1;
    if (edad >= 18 && tieneID == 1) {
        printf("Acceso permitido\n");
    } else {
        printf("Acceso denegado\n");
    }
    return 0;
}
```